GENERATIVE AI / ASSIGNMENT 2

Your assignment, explained step by step

A plain-language companion to the master plan

Understand what we are building, why each phase matters, and what you should see when it is complete.

Our working approach
Google Colab for training and experiments, subject to GPU/session availability; your laptop for the final browser application.

How to use this guide
Read for understanding, then use the matching phase in the technical master plan for implementation and checks.

Contents

How to use this guide . . . 4

First, what are we actually building? . . . 5

The big picture: the journey from start to finish . . . 6

A word about Colab and your laptop . . . 7

Phase 0 - Set up our workspaces . . . 8

Phase 1 - Learn from existing work and choose our models . . . 9

Phase 2 - Organize the dataset without giving the game away . . . 10

Phase 3 - Find out how the model performs before training . . . 11

Phase 4 - Teach the model the assignment's task . . . 12

Phase 5 - Make the model show its evidence . . . 13

Phase 6 - Help the model reuse earlier work . . . 14

Phase 7 - Understand attention and process it in smaller pieces . . . 15

Phase 8 - Let a smaller model suggest, then verify . . . 16

Phase 9 - Lock our choices and take the final test . . . 17

Phase 10 - Make it usable in a browser . . . 18

Phase 11 - Explain, package, and demonstrate the work . . . 19

The final package, in everyday terms . . . 20

How you and an agent can work together . . . 21

Compute-route update (10 November 2026): the active training route is **Google Colab**, with the laptop for development and the final app. See Colab training setup. Verify CUDA in the live session.

How to use this guide

This guide is a conversation about the project: what we are building, why each step exists, what you will see when it works, and what you should understand before moving on. Read it first when a phase in the master plan feels overwhelming.

The master plan remains the detailed instruction manual for implementation, exact tests, file names, and the grading checklist. This guide follows the same phase numbers, so you can move between the two. This is a progress guide, not the assignment's final research paper.

Our current route is to use **Google Colab** for training and research, and use your laptop for development and the final application. Colab GPU access can vary, so we verify it before training. The laptop app never requires an active Colab session.

Where we are now: Phases 0 through 8 are implemented and locally tested. We have measured the original, untrained-for-this-task model on all 206 validation examples. We have not completed the real TrialGPT adapter training or built the final app. You have chosen Google Colab for the training run. The synthetic LoRA pilot only showed that a short training procedure can run; it did not teach the model this task.

First, what are we actually building?

Imagine a researcher wants to know whether a patient's description fits the requirements for a clinical trial. A trial can have many separate requirements. Some say what a patient **must have**; others describe conditions that **would prevent** participation.

We will build a small application that reads a patient's description and checks one requirement at a time. For each requirement, it should give a category, point to the patient sentences it relied on, and explain what it found. If the information is missing, it must say so. It can also produce a simple overall screening summary using rules we clearly explain.

The application is an **educational screening prototype**. Its output is a preliminary aid for discussion, not a medical enrollment decision. The assignment also asks us to investigate how a Transformer model generates text and to implement and measure three related mechanisms: saved attention information, memory-efficient attention, and speculative decoding. We will get to each in plain language.

A small example

Suppose a made-up patient record says:

- Sentence 0: The patient is 58 years old.
- Sentence 1: The patient has type 2 diabetes.
- Sentence 2: The record does not mention kidney function.

Now consider three separate trial requirements:

Requirement	Type	What a sensible result might say
Patients must be 18 to 65 years old.	Inclusion: something required	The age requirement is met; sentence 0 supports this.
Patients with diabetes are excluded.	Exclusion: something that rules a patient out	This exclusion applies; sentence 1 supports this.
Patients must have a specified kidney function result.	Inclusion	There is not enough information in this record to decide; sentence 2 identifies what is missing.

The examples illustrate the intended behavior; they are not actual predictions from our model. The key point is that **a missing test result is not the same thing as a normal test result**. Also, meeting an exclusion condition has the opposite effect from meeting an inclusion condition. Our program has to keep that distinction clear.

The real dataset uses specific categories such as included, not included, excluded, not excluded, not enough information, and not applicable. We must preserve them instead of turning everything into a simple yes or no. We will also distinguish a model or program error from a legitimate "not enough information" answer.

The big picture: the journey from start to finish

We begin with a pre-trained model: a Transformer that has already learned general language patterns. We then teach it this particular task using expert-labeled examples. After training, we check whether its answers and evidence match examples it has not seen before. Alongside that, we implement and test the required Transformer mechanisms. Finally, we put the system behind a browser interface, write up what happened, and demonstrate it.

Think of the work as four connected stories:

1. **Prepare:** arrange our computing environments, choose the model, and organize the data fairly.
2. **Teach and check:** measure the model before training, train it, and make its answers traceable to patient sentences.
3. **Investigate:** implement and measure the three required ways of handling Transformer computation.
4. **Deliver:** run the final evaluation, build the application, write the paper, and prepare the video and live demonstration.

The phases below describe these in the same order as the master plan. For each phase, you can ask an agent to carry out the technical work, but you should still understand the purpose, inspect the output, and be able to explain what happened. The assignment is individual and explicitly allows AI assistance when it is acknowledged and understood.

A word about Colab and your laptop

Colab gives us a temporary computer in the cloud with a graphics processor, or **GPU**. GPUs can speed up model calculations. Your laptop also has a GPU, but its 6 GB of graphics memory makes model training more restrictive. Colab may have more room, although the available GPU and session duration are not guaranteed.

The Colab notebook reads a checksummed source archive from Google Drive and writes verified checkpoints and exports back to Drive. Because the session can disconnect or its temporary storage can disappear, we verify the saved files, download them, and check the hashes on your laptop. The final browser application runs locally, so the evaluator does not need Colab.

A previous Colab session ran a small synthetic pilot, but that pilot is not TrialGPT training. Kaggle's recent GPU probe received CPU-only PyTorch; those records are historical. Verify CUDA in the current Colab session before the active training run. No speedup has been measured. The detailed compute comparison is in compute strategy.

Phase 0 - Set up our workspaces

What we are doing: We are preparing two places to work: Colab for experiments and your laptop for development and the final app. An environment is simply the combination of Python and the software packages a project needs.

Why it matters: Without a repeatable setup, a program may work once and fail the next time because software versions or file locations changed. On your laptop, we also need to put large files on drive D:, where the earlier inspection found considerably more space than on drive C:.

We created the project folders, installed the local tools, and recorded what was available. In Colab, the environment report checks whether a GPU is actually assigned and how much memory it has. The notebook starts project scripts; the main implementation lives in normal source files that can also run elsewhere.

We will also practice saving and restoring a small piece of work. This is a rehearsal for an interrupted training session. If Colab's GPU is unavailable, stop and continue local work that does not need a GPU. Local assignment training remains unverified and needs a measured pilot before use.

What you should see at the end: A clear record of both environments, a working project setup, and proof that Colab Drive output can be downloaded and recovered. Interrupted-training recovery has a local rehearsal; the prior synthetic Colab pilot was not clinical training.

Explain it back: "Colab is the temporary experiment computer used through the browser notebook; my laptop is where we keep a copy of the work and run the final app."

Phase 1 - Learn from existing work and choose our models

What we are doing: We are reading credible sources about clinical trial matching, Transformers, model training, and efficient generation. We are also choosing two models: a main model to make decisions and a smaller helper model for one of the required experiments.

A **model** is the learned mathematical system that processes the text. The suggested main model is Qwen2.5-1.5B-Instruct, and the suggested helper is Qwen2.5-0.5B-Instruct. The names identify two sizes in the same model family. They are starting choices, not a guarantee that they will fit or give good results.

Why two models? The main model handles the assignment's patient-versus-criterion task. Later, the smaller model will suggest possible next words or word pieces; the main model will check them. For this to work, the two must agree on how text is broken into numbered pieces, called **tokens**. For example, a word might be one token or several. We must actually check their token rules rather than assume they match because the models share a family name.

We will do small experiments before committing: can the main model train on the verified Colab GPU, and can the laptop run both models together for the final demonstration? If the answer is no, we adjust the approach and document the decision. We also record why we chose these models over alternatives and what each source contributed.

What you should see at the end: A short research record, a justified model choice, a compatibility check for the two models, and evidence that the chosen setup fits the available machines. No final test patients are used to make these choices.

Explain it back: "We choose a model that can learn the task and still fit the final app, then verify that its smaller helper speaks in the same token IDs."

Phase 2 - Organize the dataset without giving the game away

What we are doing: We are turning the required TrialGPT annotation dataset into clean examples. Each example connects a patient description, one trial requirement, an expert's category, and the sentence numbers the expert marked as evidence.

The dataset has 53 patients. The assignment gives an exact rule for dividing them: 37 patients for training, 8 for validation, and 8 for the final test. When the current dataset was inspected using that rule, the groups contained 603, 206, and 206 individual requirement examples respectively. We will reproduce the split from the pinned dataset and publish its patient IDs and counts.

Why split by patient? If sentences from the same person appeared in both training and test, the model could benefit from familiarity with that person. Keeping every person's examples together gives a fairer check of how the model handles a new patient.

The three groups serve different purposes:

- **Training:** examples the model learns from.
- **Validation:** examples we use while making development decisions, such as choosing a saved training version.
- **Test:** examples set aside until our choices are fixed, for the final assessment.

The dataset also contains earlier AI-generated answers. Those are not the expert labels we want the model to learn from. We must keep them out of the model's inputs and the final evaluation. We retain the original patient sentence numbers so that an answer citing sentence 1 really points to the correct sentence.

Long records may not fit within the amount of text the model can process in one request. If we have to shorten a record, we will use a documented rule, check whether important evidence was removed, and report the effect. We must not use the expert answer to decide which sentences to keep.

What you should see at the end: A reproducible list of the three patient groups, clean examples, a check that no patient is in two groups, and a record of how sentence numbers and long notes are handled.

Explain it back: "The model learns from one group, we make choices using another, and we save the third for a final fair check."

Phase 3 - Find out how the model performs before training

What we did: We asked the original, untrained-for-this-task model to answer all 206 validation examples. This gives us a starting point, often called a **baseline**. We also built the scoring tools that can later judge both the original and trained model in the same way.

Why it matters: If we only show results after training, we cannot tell whether training helped. Comparing the same model before and after, on the same examples with the same answer settings, makes the change interpretable.

There are two things to score. First, did the model choose the expert's category? Second, did it cite the sentences the expert identified? For example, if the expert selected sentences 0 and 2, and the model selected 0 and 1, it found one expected sentence, missed one, and added one the expert did not select. Those counts let us measure evidence quality.

The assignment asks for several measures. **Accuracy** is the share of category answers that are correct. **Precision** asks how many answers or cited sentences in a predicted group were correct. **Recall** asks how many of the expected ones were found. **F1** balances precision and recall. **Macro-F1** averages category scores so that common answers do not completely hide poor performance on rare ones. A **confusion matrix** is a table showing which correct categories were mistaken for which other categories.

There is an important special case: sometimes neither the expert nor the model identifies an evidence sentence. The scoring code must state how it treats that case and also report results for examples where evidence was actually expected. Broken or unreadable model outputs must be counted and reported, not quietly discarded.

What we saved: All 206 answers, their raw model text, the exact prompt token IDs, per-row timing, configuration fingerprints and scores. The test data was not opened. The local model achieved accuracy 0.131 and six-category macro-F1 0.124. Its evidence micro-F1 was 0.155. It produced valid JSON on 202 of 206 examples, with no runtime failures or output-cap truncations. These are baseline software metrics, not evidence that the system is safe or clinically useful. The results are in the Phase 3 handoff.

For speed, the full run used the library's key/value cache. We checked the parsed answers against the simpler, slower decoder on three examples: the parsed answers matched, but the generated token sequences did not match exactly. The later custom-cache phase still needs to prove exact token agreement.

Explain it back: "We now have the model's before-training score, and every validation answer is saved so we can compare training fairly."

Phase 4 - Teach the model the assignment's task

What we will do next: We will show the model the training examples and adjust a small set of additional model parameters so that it learns to return the expert's category and evidence sentence numbers. This task-specific teaching is called **fine-tuning**.

The proposed method, **LoRA**, adds small trainable components while keeping most of the original model fixed. You can think of it as adding a compact task-specific adjustment to an existing model. The adjustment is usually saved as an **adapter**: a file that is loaded alongside the original model. This can be more manageable than storing and retraining an entire large model.

We start with a tiny practice run to catch mistakes. Then we train on the designated 37-patient group and periodically check the model on the validation group. We record how the training is going and choose a saved version based on validation performance. A saved training state is often called a **checkpoint**.

Because cloud sessions can fail, we must preserve resumable checkpoints during a run; notebook output alone does not prove interrupted-run recovery. Checkpoints need enough information to continue training, not just the final adapter. If the Colab GPU fails or disconnects, preserve the evidence and continue local work. Once we select a version, we download it and its records from Drive to your laptop, verify the copies, and make sure the laptop can load it.

Why validation matters here: We can use validation results to choose among checkpoints. We cannot make that choice by looking at the final test results. We also compare against the original model using the same validation examples and settings. Training may improve some measures and worsen others; we report what actually happens.

What you should see at the end: A trained adapter that loads successfully, training logs and a curve showing progress, a record of why that checkpoint was chosen, and a before/after validation comparison. We have not promised any particular score.

Explain it back: "We add a small task-specific adjustment, keep track of training, and choose the saved version using validation rather than the final test."

Phase 5 - Make the model show its evidence

What we are doing: We build the part of the system that takes the model's predicted category and sentence numbers and turns them into something a person can inspect.

If the model cites sentence 1, the program retrieves sentence 1 directly from the original patient text and displays it. The model does not get to rewrite the quotation. The explanation should be modest: it should describe the predicted category and show the cited text without inventing medical facts.

Why this matters: A confident-sounding answer can still be wrong. Displaying the exact source sentence makes it possible to check what the model relied on. A citation is useful evidence to inspect, but it is not proof that the medical reasoning is correct. If a sentence number does not exist, the system should flag it instead of making up a quotation.

For multiple requirements, we will use a written aggregation rule to make a preliminary summary. For example, a failed inclusion requirement or an applicable exclusion can produce a preliminary failure; missing information or a program error can produce "review needed." The exact rule, including how "not applicable" is handled, will be visible and documented. The dataset labels individual criteria, so this overall summary is a rule-based conclusion, not a separately verified trial-level label.

We will test examples involving missing details, contradictions, negation, and the difference between a patient's own condition and a family member's condition. We will keep a record of real mistakes to discuss in the paper.

What you should see at the end: For each requirement, a category, actual cited patient text, an honest explanation, and clear warnings when the output or evidence cannot be verified.

Explain it back: "The program shows the exact source sentences and says when it cannot support a conclusion."

Phase 6 - Help the model reuse earlier work

What we are doing: We implement two ways for the model to produce an answer one token at a time. The first repeatedly processes the full text so far. The second saves and reuses information from earlier tokens. That saved information is called the **key/value cache**, or KV cache.

Imagine writing a sentence while repeatedly rereading the whole page from the start for each new word. Remembering useful notes about what you have already read can avoid some repeated work. A KV cache plays a similar role inside the Transformer, although it stores numerical information from the model's attention layers rather than a human-style summary.

Why this is an assignment task: The examiner wants to see that we understand and implement how the saved information is passed forward and updated. Merely switching on a library setting is not enough. We will also make sure the model knows the correct position of each new token and cannot attend to future tokens.

We then ask a very concrete question: with the same model, input and deterministic settings, do the cached and uncached versions generate the **same token sequence**? We test several input and output lengths, investigate any mismatch, and measure time and memory. Deterministic here means the model always chooses its highest-scoring next token under the chosen settings.

What you should see at the end: Our own cache-aware generation code, checks showing whether the token sequences match, and measurements of how long each approach takes and how much memory it uses. We do not claim success merely because the answers look similar.

Explain it back: "The cache saves information from earlier tokens so the model can reuse it; we prove that the resulting token sequence agrees with the reference."

Phase 7 - Understand attention and process it in smaller pieces

What we are doing: We implement a basic version of **attention**, then a version that works in smaller blocks. Attention is one way a Transformer decides how much information to draw from different parts of the available text when processing a position.

For an intuitive example, when assessing "the patient has diabetes," the model may need to connect "patient" and "diabetes" with nearby words such as "does not have" or "family history." Attention is part of the model's numerical mechanism for combining context; it is not, by itself, a medical explanation.

The basic method can create a large table of relationships between positions in the text. As the text gets longer, that table becomes expensive. Our second implementation processes blocks and keeps running totals so that it does not have to hold the entire table at once. The running calculation is called **online softmax**. Softmax is the operation that turns scores into weights; the online version maintains the right totals while reading blocks one at a time.

We also enforce the rule that a position cannot look ahead at future tokens during generation. We compare both implementations on small examples and check that their numerical answers are close. Then we measure time and memory. Our educational Python implementation might be slower than a highly optimized built-in operation; the assignment values correct implementation and honest measurement, not a guaranteed speedup.

What you should see at the end: Two working attention implementations, tests across several sizes and masks, recorded numerical differences, and measured speed/memory results. Unless we actually connect this custom attention to the model, we describe it as a separate experiment.

Explain it back: "Attention combines information from available text positions; working in blocks can reduce the need to hold a huge intermediate table."

Phase 8 - Let a smaller model suggest, then verify

What we are doing: We implement **speculative decoding** using the smaller helper model and the main model. Decoding means generating the answer token by token.

Suppose the helper proposes the next four tokens. The main model checks those proposals. If the first two match what the main model would have chosen, we accept those two. If the third differs, we use the main model's choice at that point and discard the remaining unchecked proposals. Both models must then continue from the same accepted text. When all suggestions match, the main model may provide an additional token if there is room in the requested output.

This explanation uses deterministic choices: the helper and main model select their highest-scoring next tokens. The main model remains the authority over the final answer. Getting the bookkeeping right matters: after a rejected suggestion, any saved information for discarded tokens must also be removed or corrected.

Why do this? Checking a short group of proposals can sometimes be faster than making every main-model step separately. But the helper has its own cost, and rejected suggestions waste work. On this project it may be slower. We still must implement the process correctly, compare its output token by token with ordinary main-model generation, and report how many suggestions were accepted, the time, the speed, and memory use.

Our main speed comparison must use the same machine for both methods. A Colab result divided by a laptop result would mix the algorithm difference with a hardware difference and would not be a fair speedup claim. We also verify that the models fit and work in a small laptop demo.

What you should see at the end: Working draft-and-check code, tests for complete acceptance and rejection cases, evidence that final tokens match ordinary main-model output, and an honest account of whether it helped or slowed things down.

Explain it back: "The small model makes proposals; the main model checks them and corrects the first disagreement, so the final answer follows the main model."

Phase 9 - Lock our choices and take the final test

What we are doing: We write down the exact model, training checkpoint, prompt, settings, dataset split, and scoring rules we have selected. This is our frozen experiment setup. Only then do we evaluate the original and trained models on the same held-out test patients.

Why freeze first? If we repeatedly inspect test answers and adjust the model to improve them, that test stops being a fair final check. The validation group is where development choices belong. If a serious bug is discovered after test results are seen, we preserve the original results, document the correction, and explain that the revised experiment has been exposed to the test.

We report results for all test examples, including malformed outputs. We show category scores overall and separately for inclusion and exclusion criteria. We show evidence retrieval scores, examples of mistakes, and cases where information is missing or the cited evidence does not support the decision. The assignment calls for at least eight distinct errors to analyze; validation errors can supplement test errors if needed, and we must not invent errors that did not occur.

We also finish the timing and memory experiments for caching, attention, and speculation. Each timing comparison is made on the same recorded device with repeated runs. The records, tables and plots must all be saved and traceable to the actual run.

What you should see at the end: Before/after held-out results, detailed scores and confusion tables, evidence scores, an error analysis, and reproducible measurements for the required Transformer mechanisms.

Explain it back: "We decide how the system works before opening the final test, then report all results and failures honestly."

Phase 10 - Make it usable in a browser

What we are doing: We turn the completed screening pipeline into an application an evaluator can open in a browser. In our plan, the application runs on your laptop after a clearly documented setup.

The user enters a patient description, enters or chooses trial requirements, and runs the model. The application shows every criterion's category, the exact patient sentences cited, missing information, and the preliminary rule-based overall summary. It also provides a readable report the user can download.

The interface must expose meaningful comparisons of the implemented generation methods. For example, the user can run the same request through ordinary, cached, and speculative generation and see agreement and timing. The attention experiment should also be inspectable. Real measurements and live actions are more useful than pictures of results alone.

We will check what happens with empty input, overly long text, invalid sentence references, missing model files, and an out-of-memory error. A useful application explains the problem and lets the user try again. It must clearly say that it is an educational screening prototype, not a clinical enrollment decision.

What you should see at the end: You can start the app on the laptop, open it in a browser, enter a new synthetic patient, review the evidence, compare inference methods, recover from an invalid entry, and download a report. After the required files are downloaded, this should work without an active Colab session.

Explain it back: "The evaluator can use the finished system in a browser, inspect what supports each answer, and save the screening report."

Phase 11 - Explain, package, and demonstrate the work

What we are doing: We turn our records into an IEEE-format research paper prepared in LaTeX, publish a reproducible source repository, provide the trained model files, and make a recorded demonstration. We also prepare for the live defense.

The paper tells a complete story: the problem, relevant research, dataset and split, model and training, how the Transformer mechanisms work, experiments, results, mistakes, limitations, and conclusion. Figures and tables should be discussed in the text. We must describe actual experiments and acknowledge the use of AI assistants; the paper is not just a collection of screenshots.

The repository needs the source code, setup instructions, data preparation, training and evaluation scripts, implementations, app, and logs. Large model files can be hosted elsewhere, but the download link and loading instructions must work. We also provide the patient split manifest, benchmark records, the paper PDF and its source-project link, and a **5-7 minute recorded application demonstration**.

Finally, rehearse the live demo. You should be able to run an unseen example, explain a cited sentence, describe how the cache and attention work, walk through a rejected speculative suggestion, and discuss what the results do and do not show. The assignment states that missing the live demo results in no marks, so the recorded video does not replace attending it.

What you should see at the end: A working app, accessible source and model files, completed paper, logs and split manifest, a 5-7 minute video link, a submitted assignment, and a confirmed live demonstration.

Explain it back: "We are delivering a system someone can run, the evidence behind its results, and a paper and demonstration that show we understand it."

The final package, in everyday terms

Before submission, check that you can point to each of these:

- **The paper:** an IEEE-format PDF written in LaTeX, with a link to its source and a brief note on AI assistance.
- **The code:** a working GitHub repository with the app, training, evaluation and required Transformer implementations, plus setup instructions.
- **The trained model:** the saved adapter or weights, where to download them, and how to load them with the correct base model.
- **The data split:** the list of which patient IDs went into training, validation and test, with counts and checks.
- **The evidence:** training records, before/after predictions and scores, error analysis, and raw timing/memory results.
- **The application:** a browser interface that screens a patient, shows evidence, compares inference methods, and exports a readable report.
- **The demonstrations:** an accessible 5-7 minute video and attendance at the live demo.

For the detailed item-by-item grading map, use the deliverables checklist. The assignment's heading says 100 marks while its listed rubric adds up to 135; the plan covers all listed rubric items and does not assume how that discrepancy will be resolved.

How you and an agent can work together

Start with one phase at a time. Read that phase here to understand the goal. Give the matching phase of the master plan to the coding agent, along with the completed outputs it needs. Ask it to implement the work, run the listed checks, and save a short handoff explaining what it changed, what passed, and what remains uncertain.

Then review the result together. Ask the agent to show you one concrete example and explain the important code in ordinary language. If a check fails, have it investigate rather than simply marking the phase complete. Keep the final test data out of development discussions until the final evaluation phase.

At the start of a phase, you should be able to say what question it answers. At the end, you should be able to point to the output and explain why it is trustworthy. That understanding is part of the assignment and will make the live defense much easier.